

Punto 5.3 — Resolución de conflictos de concurrencia

Contexto del proyecto

Este punto continúa directamente `M05/PROYECTO/5.2`. La carpeta `M05/PROYECTO/5.3` contiene la solución completa `AceriaData.sln` y conserva las capas y migraciones existentes. El modelo oficial no cambia en este punto; se conserva la última migración real ya existente.

Objetivo práctico

Resolver `DbUpdateConcurrencyException` con políticas explícitas: cliente gana, base gana, merge, notificación, fila eliminada y reintento acotado.

Paso 1: Abrir el estado completo del punto

```
cd M05/PROYECTO/5.3
dotnet restore AceriaData.sln
```

La restauración se ejecuta sobre la solución local del punto. No es necesario copiar archivos desde otros puntos: la carpeta ya contiene el estado acumulativo completo.

Paso 2: Identificar los archivos principales

En este punto se trabajan principalmente estos archivos:

- `src/AceriaData.Infrastructure/Repositories/ResolucionConflictosM5Repositorio.cs`

Los archivos se estudian dentro de su capa real. Application conserva sus abstracciones y casos de uso; Infrastructure contiene EF Core y acceso a proveedor; Console compone y ejecuta; los tests se ubican fuera de las capas de producción cuando aparecen en 5.9 y 5.11.

Paso 3: Implementar y estudiar ResolucionConflictosM5Repositorio.cs

```
public ResolucionConflictoM5Dto ClienteGana()
{
    RestaurarOrden();
    using var scopeA = _scopeFactory.CreateScope();
    using var scopeB = _scopeFactory.CreateScope();
    var contextA = scopeA.ServiceProvider.GetRequiredService<AceriaDbContext>();
    var contextB = scopeB.ServiceProvider.GetRequiredService<AceriaDbContext>();
    var a = Cargar(contextA);
    var b = Cargar(contextB);

    a.Cliente = "Cliente A - cliente gana";
    contextA.SaveChanges();

    b.Cliente = "Cliente B - cliente gana";
    var conflicto = false;
    var intentos = 1;
    var valores = new List<ValorConflictoM5Dto>();

    try
    {
        contextB.SaveChanges();
    }
    catch (DbUpdateConcurrencyException ex)
    {
        conflicto = true;
        var entry = ex.Entries.Single();
        var db = entry.GetDatabaseValues()
            ?? throw new InvalidOperationException("La orden desapareció durante ClienteGana.");
        valores = CrearValores(entry, db);
        entry.OriginalValues.SetValues(db);
        intentos++;
        contextB.SaveChanges();
    }

    var final = LeerFinal();
    return Resultado("Cliente gana", conflicto, final, intentos, valores);
}
```

Explicación línea a línea — ResolucionConflictosM5Repositorio.cs

Línea 1: `public ResolucionConflictoM5Dto ClienteGana()` → Declara la operación pública que ejecutará o verificará este comportamiento.

Línea 2: `{` → Delimita el bloque sintáctico correspondiente.

Línea 3: `RestaurarOrden();` → Ejecuta esta instrucción como parte del flujo del ejemplo.

Línea 4: `using var scopeA = _scopeFactory.CreateScope();` → Crea un ámbito de DI independiente; los servicios Scoped, incluido `AceriaDbContext`, no se comparten con otros actores.

Línea 5: `using var scopeB = _scopeFactory.CreateScope();` → Crea un ámbito de DI independiente; los servicios Scoped, incluido `AceriaDbContext`, no se comparten con otros actores.

Línea 6: `var contextA = scopeA.ServiceProvider.GetRequiredService<AceriaDbContext>();` → Resuelve el `DbContext` del ámbito y garantiza que la configuración de Infrastructure sea la misma que en la aplicación.

Línea 7: `var contextB = scopeB.ServiceProvider.GetRequiredService<AceriaDbContext>();` → Resuelve el `DbContext` del ámbito y garantiza que la configuración de Infrastructure sea la misma que en la aplicación.

Línea 8: `var a = Cargar(contextA);` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 9: `var b = Cargar(contextB);` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 11: `a.Cliente = "Cliente A - cliente gana";` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 12: `contextA.SaveChanges();` → Envía al proveedor los cambios rastreados y aplica las reglas de transacción/concurrencia configuradas.

Línea 14: `b.Cliente = "Cliente B - cliente gana";` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 15: `var conflicto = false;` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 16: `var intentos = 1;` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 17: `var valores = new List<ValorConflictoM5Dto>();` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 19: `try` → Abre el bloque en el que se espera que pueda producirse la excepción que se quiere estudiar.

Línea 20: `{` → Delimita el bloque sintáctico correspondiente.

Línea 21: `contextB.SaveChanges();` → Envía al proveedor los cambios rastreados y aplica las reglas de transacción/concurrencia configuradas.

Línea 22: `}` → Delimita el bloque sintáctico correspondiente.

Línea 23: `catch (DbUpdateConcurrencyException ex)` → Captura la excepción específica para aplicar la política de resolución prevista.

Línea 24: `{` → Delimita el bloque sintáctico correspondiente.

Línea 25: `conflicto = true;` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 26: `var entry = ex.Entries.Single();` → Materializa o agrega la consulta en este punto, provocando la ejecución contra el proveedor cuando procede.

Línea 27: `var db = entry.GetDatabaseValues();` → Obtiene de la base el estado actual de la fila tras detectar el conflicto.

Línea 28: `?? throw new InvalidOperationException("La orden desapareció durante ClienteGana.");` → Ejecuta esta instrucción como parte del flujo del ejemplo.

Línea 29: `valores = CrearValores(entry, db);` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 30: `entry.OriginalValues.SetValues(db);` → Actualiza los valores originales del entry para reintentar contra la versión recién leída.

Línea 31: `intentos++;` → Ejecuta esta instrucción como parte del flujo del ejemplo.

Línea 32: `contextB.SaveChanges();` → Envía al proveedor los cambios rastreados y aplica las reglas de transacción/concurrencia configuradas.

Línea 33: `}` → Delimita el bloque sintáctico correspondiente.

Línea 35: `var final = LeerFinal();` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 36: `return Resultado("Cliente gana", conflicto, final, intentos, valores);` → Devuelve la evidencia recopilada por el escenario para que el caso de uso o el test pueda comprobarla.

Línea 37: `}` → Delimita el bloque sintáctico correspondiente.

Paso 4: Implementar y estudiar ResolucionConflictosM5Repositorio.cs

```
public ResolucionConflictoM5Dto ResolucionPersonalizada()
{
    RestaurarOrden();
    using var scopeA = _scopeFactory.CreateScope();
    using var scopeB = _scopeFactory.CreateScope();
    var contextA = scopeA.ServiceProvider.GetRequiredService<AceriaDbContext>();
    var contextB = scopeB.ServiceProvider.GetRequiredService<AceriaDbContext>();
    var a = Cargar(contextA);
    var b = Cargar(contextB);

    a.Cliente = "Cliente A - merge";
    a.Estado = "EnProceso A";
    contextA.SaveChanges();

    b.Cliente = "Cliente B - merge";
    b.Estado = "Completada B";
    var intentos = 1;

    try
    {
        contextB.SaveChanges();
    }
    catch (DbUpdateConcurrencyException ex)
    {
        var entry = ex.Entries.Single();
        var db = entry.GetDatabaseValues()
            ?? throw new InvalidOperationException("La orden desapareció durante la fusión.");
        entry.OriginalValues.SetValues(db);
        entry.CurrentValues[nameof(OrdenFabricacion.Estado)] = db[nameof(OrdenFabricacion.Estado)];
        intentos++;
        contextB.SaveChanges();
    }

    return Resultado("Resolución personalizada", true, LeerFinal(), intentos, []);
}
```

Explicación línea a línea — ResolucionConflictosM5Repositorio.cs

Línea 1: `public ResolucionConflictoM5Dto ResolucionPersonalizada()` → Declara la operación pública que ejecutará o verificará este comportamiento.

Línea 2: `{` → Delimita el bloque sintáctico correspondiente.

Línea 3: `RestaurarOrden();` → Ejecuta esta instrucción como parte del flujo del ejemplo.

Línea 4: `using var scopeA = _scopeFactory.CreateScope();` → Crea un ámbito de DI independiente; los servicios Scoped, incluido `AceriaDbContext`, no se comparten con otros actores.

Línea 5: `using var scopeB = _scopeFactory.CreateScope();` → Crea un ámbito de DI independiente; los servicios Scoped, incluido `AceriaDbContext`, no se comparten con otros actores.

Línea 6: `var contextA = scopeA.ServiceProvider.GetRequiredService<AceriaDbContext>();` → Resuelve el `DbContext` del ámbito y garantiza que la configuración de Infrastructure sea la misma que en la aplicación.

Línea 7: `var contextB = scopeB.ServiceProvider.GetRequiredService<AceriaDbContext>();` → Resuelve el `DbContext` del ámbito y garantiza que la configuración de Infrastructure sea la misma que en la aplicación.

Línea 8: `var a = Cargar(contextA);` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 9: `var b = Cargar(contextB);` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 11: `a.Cliente = "Cliente A - merge";` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 12: `a.Estado = "EnProceso A";` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 13: `contextA.SaveChanges();` → Envía al proveedor los cambios rastreados y aplica las reglas de transacción/concurrencia configuradas.

Línea 15: `b.Cliente = "Cliente B - merge";` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 16: `b.Estado = "Completada B";` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 17: `var intentos = 1;` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 19: `try` → Abre el bloque en el que se espera que pueda producirse la excepción que se quiere estudiar.

Línea 20: `{` → Delimita el bloque sintáctico correspondiente.

Línea 21: `contextB.SaveChanges();` → Envía al proveedor los cambios rastreados y aplica las reglas de transacción/concurrencia configuradas.

Línea 22: `}` → Delimita el bloque sintáctico correspondiente.

Línea 23: `catch (DbUpdateConcurrencyException ex)` → Captura la excepción específica para aplicar la política de resolución prevista.

Línea 24: `{` → Delimita el bloque sintáctico correspondiente.

Línea 25: `var entry = ex.Entries.Single();` → Materializa o agrega la consulta en este punto, provocando la ejecución contra el proveedor cuando procede.

Línea 26: `var db = entry.GetDatabaseValues();` → Obtiene de la base el estado actual de la fila tras detectar el conflicto.

Línea 27: `?? throw new InvalidOperationException("La orden desapareció durante la fusión.");` → Ejecuta esta instrucción como parte del flujo del ejemplo.

Línea 28: `entry.OriginalValues.SetValues(db);` → Actualiza los valores originales del entry para reintentar contra la versión recién leída.

Línea 29: `entry.CurrentValues[nameof(OrdenFabricacion.Estado)] = db[nameof(OrdenFabricacion.Estado)];` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 30: `intentos++;` → Ejecuta esta instrucción como parte del flujo del ejemplo.

Línea 31: `contextB.SaveChanges();` → Envía al proveedor los cambios rastreados y aplica las reglas de transacción/concurrencia configuradas.

Línea 32: `}` → Delimita el bloque sintáctico correspondiente.

Línea 34: `return Resultado("Resolución personalizada", true, LeerFinal(), intentos, []);` → Devuelve la evidencia recopilada por el escenario para que el caso de uso o el test pueda comprobarla.

Línea 35: `}` → Delimita el bloque sintáctico correspondiente.

Paso 5: Implementar y estudiar ResolucionConflictosM5Repositorio.cs

```
while (true)
{
    intentos++;
    try
    {
        contextB.SaveChanges();
        break;
    }
    catch (DbUpdateConcurrencyException ex) when (intentos < maxIntentos)
    {
        conflicto = true;
        var entry = ex.Entries.Single();
        var db = entry.GetDatabaseValues()
        ?? throw new InvalidOperationException("La fila ya no existe.");
        entry.OriginalValues.SetValues(db);
    }
}
```

Explicación línea a línea — ResolucionConflictosM5Repositorio.cs

Línea 1: `while (true)` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 2: `{` → Delimita el bloque sintáctico correspondiente.

Línea 3: `intentos++;` → Ejecuta esta instrucción como parte del flujo del ejemplo.

Línea 4: `try` → Abre el bloque en el que se espera que pueda producirse la excepción que se quiere estudiar.

Línea 5: `{` → Delimita el bloque sintáctico correspondiente.

Línea 6: `contextB.SaveChanges();` → Envía al proveedor los cambios rastreados y aplica las reglas de transacción/concurrencia configuradas.

Línea 7: `break;` → Ejecuta esta instrucción como parte del flujo del ejemplo.

Línea 8: `}` → Delimita el bloque sintáctico correspondiente.

Línea 9: `catch (DbUpdateConcurrencyException ex) when (intentos < maxIntentos)` → Captura la excepción específica para aplicar la política de resolución prevista.

Línea 10: `{` → Delimita el bloque sintáctico correspondiente.

Línea 11: `conflicto = true;` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 12: `var entry = ex.Entries.Single();` → Materializa o agrega la consulta en este punto, provocando la ejecución contra el proveedor cuando procede.

Línea 13: `var db = entry.GetDatabaseValues()` → Obtiene de la base el estado actual de la fila tras detectar el conflicto.

Línea 14: `?? throw new InvalidOperationException("La fila ya no existe.");` → Ejecuta esta instrucción como parte del flujo del ejemplo.

Línea 15: `entry.OriginalValues.SetValues(db);` → Actualiza los valores originales del entry para reintentar contra la versión recién leída.

Línea 16: `}` → Delimita el bloque sintáctico correspondiente.

Línea 17: `}` → Delimita el bloque sintáctico correspondiente.

Paso 6: Compilar y ejecutar

```
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
dotnet ef migrations has-pending-model-changes --project src/AceriaData.Infrastructure --startup-
project src/AceriaData.Console --configuration Release
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release --no-build
```

La ejecución debe finalizar con **5.3 OK**. Si una comprobación interna falla, la aplicación lanza una excepción y el punto no se considera correctamente reproducido.

Diagnóstico técnico

La práctica se considera correcta cuando se observan estas condiciones:

- `GetDatabaseValues` recupera el estado actual de SQL Server.
- `OriginalValues.SetValues` actualiza la versión contra la que se reintentará.
- `Reload` descarta cambios locales cuando la base gana.
- La resolución personalizada decide propiedad por propiedad.
- Los reintentos están limitados.

Reto resuelto y ampliación

Reto: Añade una política que preserve Cliente local, Estado de base y combine Observaciones solo cuando ambos lados no sean nulos.

Para resolverlo, conserva la misma regla del módulo: primero define qué resultado funcional debe mantenerse; después captura SQL, número de comandos, tracking, historial de migraciones o estado transaccional según corresponda. No uses una medición temporal aislada como sustituto de la evidencia técnica.

Errores comunes

- Reintentar indefinidamente.
- Confundir `CurrentValues`, `OriginalValues` y valores actuales de base.
- Sobrescribir automáticamente sin informar al usuario cuando el negocio requiere decisión.
- No tratar `GetDatabaseValues()` == null cuando la fila fue eliminada.

Cierre acumulativo

1. Ejecuta `dotnet build` en la solución del punto.
2. Ejecuta la aplicación y comprueba el marcador **5.3 OK**.
3. Conserva las migraciones existentes; solo 5.2 añade el cambio de modelo oficial de M5.
4. Comprueba que `Application` continúa sin depender de Entity Framework Core ni de Infrastructure.
5. Antes de avanzar, relaciona el resultado con el SQL, la transacción, el historial o la prueba que lo demuestra.

Resultado esperado

Al finalizar el punto 5.3, `AceriaData` debe compilar, ejecutarse sobre SQL Server LocalDB y demostrar de forma observable el comportamiento descrito. El marcador final es:

5.3 OK

Conexión con el siguiente punto

El siguiente estado es 5.4 y parte directamente de este proyecto completo.